

Summer 2026 REU Code Walkthrough

Every file, every experiment, and where the numbers come from

Repository: /Users/davidvalenta/deep-wind-model-dispatch

Use this as the guide during meetings: start from the folder runner, then follow the RUNNER or SOURCE_RUNNER variable into the code folder.

Generated: 2026-07-27 10:09

This guide explains the Summer 2026 REU folder as a reviewer-facing code map. It is written for someone who knows the project idea but needs to know where each result is produced, which file to run, which knobs change the experiment, and which lines contain the important equations.

The Project in One Page

The Summer 2026 folder is organized as a ladder. Each step adds one idea and saves its own results:

Step	Folder	Main question	Main command
0	`100 MW baseload`	What does Chris's rule-based 100 MW storage benchmark do?	<code>`../../venv/bin/python RUN_0_100MW_BASELOAD.py`</code>
1	`causal ridge regression`	Which forecast method predicts wind power best?	<code>`../../venv/bin/python RUN_1_FORECAST_RMSE.py`</code>
2	`rolling horizon`	If we use the causal ridge forecast, which Gurobi planning horizon works best?	<code>`../../venv/bin/python RUN_2_ROLLING_HORIZON.py`</code>
3	`different scenarios`	Does using several possible futures improve dispatch over one forecast?	<code>`../../venv/bin/python RUN_3_SCENARIO_COMPARISON.py`</code>
4	`oracle upper bound`	What is the ceiling if Gurobi knows the true future?	<code>`../../venv/bin/python RUN_4_ORACLE_UPPER_BOUND.py`</code>

The repeated pattern in every experiment is:

1. Edit EXPERIMENT_KNOBS.py in that folder if you want to change storage power, duration, horizon, output folder, or rerun mode.
2. Run the folder's RUN_...py file.
3. The runner calls the deeper code in the local code/ folder.
4. Results go into results/; figures go into figures/.

Current Baselines: Do Not Mix These Up

Name	Meaning	Storage?	Used for
Wind-only / no storage	Actual wind is delivered directly up to the 249 MW grid cap; excess wind is curtailed.	No	Main clean physical reference.
Older storage baseload	A previous non-strategic storage smoothing rule from earlier project code.	Yes	Historical context only.
100 MW constant-output baseload	Chris's rule-based benchmark that tries to hold 100 MW output	Yes	Required side benchmark.

	using 100 MW / 10 h CAES.		
Gurobi dispatch	MILP chooses charge/direct/discharge timing to maximize revenue subject to constraints.	Yes	Proposed optimization controller.
Oracle	Same Gurobi dispatch, but with realized future wind and price.	Yes	Upper bound, not deployable.

Main comparison in the corrected terminal output is wind-only. The 100 MW benchmark is printed underneath as side information because it is a different storage rule, not the no-storage baseline.

Current Key Numbers Saved in the Repo

Step 0: 100 MW Constant-Output Baseload, 2014-2023

- Revenue metric: 5,981,942.95
- Raw revenue USD: \$211,515,621.83
- Final SoC: 980.22 MWh
- SoC range: 200.00 to 1000.00 MWh

Step 1: Forecast RMSE Ranking

Rank	Model	RMSE MW	MAE MW	Bias MW
1	`causal_lag_prediction_mw`	21.24	13.62	-0.18
2	`lag1_persistence_prediction_mw`	23.60	14.55	0.00
3	`speed_power_curve_prediction_mw`	41.86	30.79	-1.34
4	`rnn_preds`	46.21	33.31	-3.62
5	`physics_preds`	50.85	36.49	8.59
6	`prob_preds`	71.69	50.42	-0.73

Step 2: Causal Ridge + Daily Rolling-Horizon Gurobi

Horizon	COVE	COVE gain vs wind-only	Revenue metric	Raw revenue gain vs wind-only	COVE gain vs 100 MW
24 h	6.966281	-30.43%	7,380,799.56	-7.10%	18.95%
48 h	6.822045	-27.73%	7,536,849.56	-4.12%	20.63%
72 h	6.830033	-27.88%	7,528,034.19	-4.33%	20.54%
168 h	6.847708	-28.21%	7,508,603.24	-4.67%	20.33%

Step 3: Scenario Dispatch

Method	Revenue	Revenue gain vs wind-only	COVE	COVE gain vs wind-only	COVE gain vs 100 MW
1 forecast	337,322,348.04	21.19%	0.173884	-13.72%	37.23%
3 scenarios	353,949,333.45	27.16%	0.165716	-8.38%	40.18%
5 scenarios	353,117,910.43	26.86%	0.166106	-8.64%	40.04%
7 scenarios	353,220,656.50	26.90%	0.166058	-8.60%	40.05%
10 scenarios	341,858,797.71	22.82%	0.171577	-12.21%	38.06%

Step 4: Daily-Replan Oracle

Horizon	COVE	COVE gain vs wind-only	Revenue metric	Raw revenue gain vs wind-only	COVE gain vs 100 MW
24 h	5.236266	1.96%	9,819,350.07	24.89%	39.08%
48 h	5.104091	4.43%	10,073,630.57	30.49%	40.62%
72 h	5.084378	4.80%	10,112,687.75	29.91%	40.85%
168 h	5.082358	4.84%	10,116,705.90	30.86%	40.87%

Step 4 Extra: Hourly-Replan Oracle Ceiling

- 168 h hourly-replan oracle: COVE 5.076786, COVE gain vs wind-only 4.84%, revenue metric 10,127,810.67, final SoC 200.00 MWh.

How the Code Passes Knobs Into the Real Model

Each step has two layers:

- EXPERIMENT_KNOBS.py: simple variables humans edit, such as STORAGE_DURATION_H = 10.0 or HORIZONS = [24, 48, 72, 168].
- RUN_...py: builds a command list called cmd, then calls the deeper model with subprocess.run(...).

A typical command list looks like this in Python:

```
cmd = [sys.executable, str(SOURCE_RUNNER), "--storage-power-mw",  
       str(knobs.STORAGE_POWER_MW), ...]  
subprocess.run(cmd, cwd=REPO_ROOT, check=True)
```

That means the visible RUN_...py file is mostly a clean front door. The heavy optimization happens inside the deeper files in code/.

File-by-File Guide

Summer 2026 REU/100 MW baseload/EXPERIMENT_KNOBS.py

- Total lines: 44
- Purpose: Stores the 100 MW benchmark knobs: storage power, duration, RTE, min/max/initial SoC, target output, grid cap, and output folders.
- Important: Change these values if Chris asks for a different benchmark setup.
- Important: This file does not run optimization by itself.

Summer 2026 REU/100 MW baseload/RUN_0_100MW_BASELOAD.py

- Total lines: 388
- Purpose: Front-door runner for Step 0. It runs the 2020 canonical benchmark and the 2014-2023 full-period 100 MW benchmark, then compares other methods to the 100 MW benchmark.
- Important: rerun_from_knobs builds the 2020 command.
- Important: rerun_full_period_baseoad_from_knobs builds the 2014-2023 command.
- Important: main prints the benchmark summary and saves comparison CSVs/figures.

Function	Lines	What it does
<code>`read_rows`</code>	69-73	Helper function.
<code>`pct_gain`</code>	76-77	Helper function.
<code>`pct_reduction`</code>	80-81	Helper function.
<code>`add_optional`</code>	84-86	Helper function.

`rerun_from_knobs`	89-119	Builds the terminal command that calls the deeper runner.
`rerun_full_period_baseload_from_knobs`	122-161	Builds the terminal command that calls the deeper runner.
`main`	164-383	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/100 MW baseload/code/build_100mw_baseload_reference.py

- Total lines: 406
- Purpose: Builds the 2014-2023 100 MW constant-output side benchmark used under Steps 2, 3, and 4.
- Important: This is the full-period version of Chris's 100 MW rule.
- Important: It writes hourly CSVs and summary CSVs for comparison.

Class	Lines	What it represents
`StorageConfig`	33-61	Storage configuration and derived storage properties.

Function	Lines	What it does
`capacity_mwh`	46-47	Helper function.
`min_soc`	50-51	Helper function.
`max_soc`	54-55	Helper function.
`initial_soc`	58-61	Helper function.
`load_data`	64-95	Reads data/configuration from CSV/YAML/text files.
`run_100mw_baseload`	98-143	Reads data/configuration from CSV/YAML/text files.
`summarize_period`	146-169	Computes result rows: revenue, COVE, final SoC, and QA fields.
`add_period_100mw_comparison`	172-218	Helper function.
`make_figures`	221-272	Creates or formats figures.
`main`	275-401	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/100 MW baseload/code/canonical_benchmark_oracle_runner.py

- Total lines: 696
- Purpose: Implements Chris's exact 2020 100 MW benchmark and canonical oracle MILP cases.
- Important: run_constant_output_baseload is the exact rule: if wind $\geq$ 100 MW, deliver 100 MW and charge excess; if wind < 100 MW, discharge to fill the deficit.
- Important: solve_oracle_window is the Gurobi MILP for perfect-information dispatch.
- Important: qa_for_labels checks constraints and SoC recursion.

Class	Lines	What it represents
`StorageConfig`	48-81	Storage configuration and derived storage properties.

Function	Lines	What it does
`capacity_mwh`	66-67	Helper function.
`annualized_cost_usd`	70-81	Computes revenue, COVE, cost, or percentage improvement.
`git_value`	84-88	Helper function.
`parse_power_file`	91-101	Reads data/configuration from

		CSV/YAML/text files.
`load_2020_pyron_rtm`	104-173	Reads data/configuration from CSV/YAML/text files.
`compute_cove`	176-179	Computes revenue, COVE, cost, or percentage improvement.
`run_constant_output_baseload`	182-241	Reads data/configuration from CSV/YAML/text files.
`solve_oracle_window`	244-308	Builds/solves the optimization problem for one planning window.
`run_oracle_rolling_horizon`	311-389	Helper function.
`chronological_continuity_error`	392-395	Helper function.
`qa_for_labels`	398-463	Checks feasibility: SoC bounds, grid cap, wind balance, and recursion.
`summarize_case`	466-499	Computes result rows: revenue, COVE, final SoC, and QA fields.
`write_hourly`	502-507	Helper function.
`write_registry`	510-555	Helper function.
`main`	558-691	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/b6 verification/code/B6_CANONICAL_RUNNER.py

- Total lines: 768
- Purpose: Frozen B6 verification runner requested by Chris.
- Important: Separate from the paper ladder.
- Important: Runs A/B/C Oracle/Causal 2020 verification cases.

Class	Lines	What it represents
`ForecastModel`	27-37	A fitted forecast model used to predict future wind/price/power values.

Function	Lines	What it does
`predict`	34-37	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`git_value`	40-41	Helper function.
`parse_power_file`	44-50	Reads data/configuration from CSV/YAML/text files.
`parse_raw_imp_file`	53-64	Reads data/configuration from CSV/YAML/text files.
`load_b6_data`	67-139	Reads data/configuration from CSV/YAML/text files.
`calendar_features`	142-156	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`origin_features`	159-171	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`single_origin_features`	174-186	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`fit_direct_models`	189-218	Helper function.
`make_forecasts`	221-236	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`prepare_forecast_frame`	239-287	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`solve_dispatch_window`	290-368	Builds/solves the optimization problem for one planning window.

<code>`execute_plan`</code>	371-418	Converts the planned action into realized operation using actual wind and current SoC.
<code>`qa_checks`</code>	421-459	Checks feasibility: SoC bounds, grid cap, wind balance, and recursion.
<code>`discharge_loss`</code>	462-464	Helper function.
<code>`run_oracle`</code>	467-502	Helper function.
<code>`run_causal`</code>	505-579	Helper function.
<code>`summarize`</code>	582-631	Computes result rows: revenue, COVE, final SoC, and QA fields.
<code>`write_hourly`</code>	634-653	Helper function.
<code>`main`</code>	656-763	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/b6 verification/code/B6_FINAL_VALIDATE.py

- Total lines: 114
- Purpose: QA validator for the B6 package.
- Important: Checks the final B6 files and summaries.

Function	Lines	What it does
<code>`validate`</code>	33-101	Checks feasibility: SoC bounds, grid cap, wind balance, and recursion.
<code>`main`</code>	104-109	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/causal ridge regression/EXPERIMENT_KNOBS.py

- Total lines: 28
- Purpose: Stores Step 1 forecast comparison knobs: data file, output file, and train/test split settings.
- Important: This is where Step 1 knows which processed data file to read.

Summer 2026 REU/causal ridge regression/RUN_1_FORECAST_RMSE.py

- Total lines: 171
- Purpose: Front-door runner for Step 1. It rebuilds or reads the forecast RMSE comparison and makes figures.
- Important: It ranks causal lag/ridge against persistence, power curve, RNN outputs, physics, and probabilistic predictions.

Function	Lines	What it does
<code>`load_rows`</code>	42-46	Reads data/configuration from CSV/YAML/text files.
<code>`rebuild_rmse_table`</code>	49-68	Helper function.
<code>`main`</code>	71-166	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/causal ridge regression/code/causal_lag_forecast.py

- Total lines: 197
- Purpose: Builds the causal lag/ridge forecast model.
- Important: `build_causal_features` creates features from present/past values only.
- Important: `fit_ridge` fits the ridge regression formula.
- Important: `evaluate_predictions` computes RMSE, MAE, and bias.

Function	Lines	What it does
`rmse`	39-40	Helper function.
`mae`	43-44	Helper function.
`chronological_slices`	47-54	Helper function.
`build_causal_features`	57-103	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`fit_ridge`	106-109	Helper function.
`fit_speed_power_curve`	112-114	Helper function.
`predict_speed_power_curve`	117-119	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`evaluate_predictions`	122-137	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`main`	140-192	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/causal ridge regression/code/compare_forecast_rmse.py

- Total lines: 131
- Purpose: Compares several power prediction methods and writes the ranked RMSE table.
- Important: This is where the final Step 1 forecast ranking comes from.

Function	Lines	What it does
`error_row`	27-44	Helper function.
`build_causal_predictions`	47-62	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`main`	65-126	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/different scenarios/EXPERIMENT_KNOBS.py

- Total lines: 46
- Purpose: Stores Step 3 uncertainty/scenario knobs.
- Important: Key knob: HORIZON_HOURS = 48.
- Important: Key knob: VARIANTS, where you choose single, 3, 5, 7, or 10 scenario runs.

Summer 2026 REU/different scenarios/RUN_3_SCENARIO_COMPARISON.py

- Total lines: 350
- Purpose: Front-door runner for Step 3 scenario comparison.
- Important: Prints wind-only main comparison, older storage-baseload context, and 100 MW side benchmark.
- Important: Builds enriched summary CSV and figures.

Function	Lines	What it does
`annualized_wind_only_cost`	52-53	Computes revenue, COVE, cost, or percentage improvement.
`load_rows`	56-60	Reads data/configuration from CSV/YAML/text files.
`short_name`	63-71	Helper function.
`money`	74-75	Helper function.
`add_wind_only_columns`	78-96	Helper function.
`add_100mw_side_columns`	99-120	Helper function.
`add_optional`	123-125	Helper function.
`rerun_from_knobs`	128-165	Builds the terminal command that calls the

		deeper runner.
`main`	168-345	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/different scenarios/code/run_best_forecast_dispatch_search.py

- Total lines: 497
- Purpose: Helper for candidate summaries and forecast/dispatch comparison figures.
- Important: Computes wind-only revenue, older baseload revenue, dispatch revenue, COVE, and gains.

Function	Lines	What it does
`matrix_from_indices`	29-35	Helper function.
`actual_future_matrix`	38-39	Helper function.
`hourly_climatology_matrix`	42-63	Helper function.
`clip_generation`	66-67	Helper function.
`clip_price`	70-71	Helper function.
`cove_reduction_from_revenues`	74-75	Computes revenue, COVE, cost, or percentage improvement.
`annualized_wind_only_cost`	78-82	Computes revenue, COVE, cost, or percentage improvement.
`candidate_summary`	85-134	Computes result rows: revenue, COVE, final SoC, and QA fields.
`evaluate_forecasts`	137-148	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`legacy_power_model_metrics`	151-198	Helper function.
`make_figures`	201-286	Creates or formats figures.
`main`	289-492	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/different scenarios/code/run_nora_matching_forecast_horizons.py

- Total lines: 776
- Purpose: Support code file in the Summer 2026 REU folder.

Class	Lines	What it represents
`DirectForecastModel`	50-60	A fitted forecast model used to predict future wind/price/power values.

Function	Lines	What it does
`predict`	57-60	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`calendar_features`	63-77	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`origin_features`	80-92	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`single_origin_features`	95-107	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`fit_direct_models`	110-148	Helper function.
`make_generation_forecasts`	151-168	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`make_weekly_price_forecasts`	171-177	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`forecast_metrics`	180-207	Builds forecast inputs, fits a forecast model, or creates predicted future values.

`solve_window_nora`	210-258	Builds/solves the optimization problem for one planning window.
`execute_frozen_day`	261-314	Converts the planned action into realized operation using actual wind and current SoC.
`run_forecast_horizon`	317-393	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`revenue`	396-397	Computes revenue, COVE, cost, or percentage improvement.
`annualized_dispatch_cost`	400-403	Computes revenue, COVE, cost, or percentage improvement.
`continuous_baseload`	406-423	Reads data/configuration from CSV/YAML/text files.
`check_realized_constraints`	426-445	Checks feasibility: SoC bounds, grid cap, wind balance, and recursion.
`summarize`	448-484	Computes result rows: revenue, COVE, final SoC, and QA fields.
`validate_nora_week`	487-502	Checks feasibility: SoC bounds, grid cap, wind balance, and recursion.
`make_figures`	505-647	Creates or formats figures.
`main`	650-771	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/different scenarios/code/run_scenario_calibration_search.py

- Total lines: 255
- Purpose: Extra scenario calibration search tool.
- Important: Used for testing scenario settings; not the primary paper result.

Function	Lines	What it does
`normalize`	25-27	Helper function.
`make_specs`	30-97	Helper function.
`configure_base`	100-112	Creates or formats figures.
`build_context`	115-125	Helper function.
`select_period`	128-141	Helper function.
`run_one`	144-167	Helper function.
`main`	170-251	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/different scenarios/code/run_uncertainty_aware_dispatch.py

- Total lines: 750
- Purpose: Main uncertainty-aware scenario controller.
- Important: Builds residual-based wind/price scenarios.
- Important: Solves a multi-scenario Gurobi MILP.
- Important: Enforces first-hour non-anticipativity, meaning all futures share the first action.
- Important: Executes only the first hour, then repeats.

Function	Lines	What it does
`matrix_from_indices`	70-75	Helper function.
`build_forecasts`	78-99	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`conformal_quantile`	102-108	Helper function.
`residual_quantiles`	111-135	Helper function.
`scenario_matrices`	138-153	Helper function.

`solve_scenario_window`	156-223	Builds/solves the optimization problem for one planning window.
`execute_first_hour_storage_action`	226-260	Converts the planned action into realized operation using actual wind and current SoC.
`baseline_value_for_scenarios`	263-295	Helper function.
`baseline_first_hour_action`	298-330	Helper function.
`run_single_forecast_recourse`	333-391	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`run_scenario_controller`	394-455	Helper function.
`make_label_row`	458-489	Helper function.
`summarize`	492-501	Computes result rows: revenue, COVE, final SoC, and QA fields.
`make_figures`	504-527	Creates or formats figures.
`main`	530-745	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/oracle upper bound/EXPERIMENT_KNOBS.py

- Total lines: 59
- Purpose: Stores Step 4 oracle knobs.
- Important: Daily oracle uses execution/replanning 24 h.
- Important: Hourly oracle ceiling uses execution/replanning 1 h.
- Important: Both use 100 MW / 10 h CAES.

Summer 2026 REU/oracle upper bound/RUN_4_ORACLE_UPPER_BOUND.py

- Total lines: 343
- Purpose: Front-door runner for oracle upper bound.
- Important: Prints two separate blocks: daily-replan oracle and hourly-replan oracle ceiling.
- Important: Uses wind-only as main comparison and 100 MW benchmark as side information.

Function	Lines	What it does
`add_optional`	44-46	Helper function.
`command`	49-100	Builds the terminal command that calls the deeper runner.
`load_rows`	103-105	Reads data/configuration from CSV/YAML/text files.
`copy_outputs`	108-111	Helper function.
`save_oracle_only`	114-119	Helper function.
`cove_gain_vs_wind`	122-123	Computes revenue, COVE, cost, or percentage improvement.
`cove_gain_vs_100mw`	126-128	Computes revenue, COVE, cost, or percentage improvement.
`raw_revenue_gain_vs_wind`	131-133	Computes revenue, COVE, cost, or percentage improvement.
`raw_revenue_gain_vs_100mw`	136-138	Computes revenue, COVE, cost, or percentage improvement.
`draw_figures`	141-214	Creates or formats figures.
`print_oracle_block`	217-264	Helper function.
`run_or_read`	267-281	Helper function.
`main`	284-338	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/oracle upper bound/code/build_oracle_summary.py

- Total lines: 67
- Purpose: Small helper that extracts oracle rows into a clean summary.
- Important: Useful after oracle runs if you want only oracle rows.

Function	Lines	What it does
`main`	27-62	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/oracle upper bound/code/dataset.py

- Total lines: 44
- Purpose: Support code file in the Summer 2026 REU folder.

Class	Lines	What it represents
`VF2Dataset`	4-25	Old neural-network model class retained from the original project code.
`VFDataset`	27-44	Old neural-network model class retained from the original project code.

Function	Lines	What it does
`__init__`	5-9	Helper function.
`__len__`	12-20	Helper function.
`__getitem__`	23-25	Helper function.
`__init__`	28-31	Helper function.
`__len__`	34-39	Helper function.
`__getitem__`	42-44	Helper function.

Summer 2026 REU/oracle upper bound/code/forecast_backtest_rolling_horizons.py

- Total lines: 1315
- Purpose: Oracle copy of the rolling-horizon forecast backtest engine.
- Important: Same structure as Step 2, but Step 4 calls it with --oracle-only.

Class	Lines	What it represents
`DirectForecastModel`	56-66	A fitted forecast model used to predict future wind/price/power values.

Function	Lines	What it does
`predict`	63-66	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`calendar_features`	69-83	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`origin_features`	86-98	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`single_origin_features`	101-114	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`fit_direct_models`	117-168	Helper function.
`make_forecast_matrix`	171-196	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`make_known_future_matrix`	199-210	Helper function.
`forecast_metrics`	213-251	Builds forecast inputs, fits a forecast model, or creates predicted future values.

`execute_plan_against_actual`	254-315	Converts the planned action into realized operation using actual wind and current SoC.
`check_realized_constraints`	318-383	Checks feasibility: SoC bounds, grid cap, wind balance, and recursion.
`year_end_soc_targets`	386-400	Helper function.
`annual_soc_qa`	403-423	Checks feasibility: SoC bounds, grid cap, wind balance, and recursion.
`wind_only_delivery`	426-428	Helper function.
`constant_output_100mw_delivery`	431-476	Helper function.
`cost_over_revenue`	479-482	Computes revenue, COVE, cost, or percentage improvement.
`apply_direct_reserve`	485-509	Helper function.
`run_horizon`	512-746	Helper function.
`style_axis`	749-753	Creates or formats figures.
`save_figures`	756-930	Creates or formats figures.
`main`	933-1310	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/oracle upper bound/code/model.py

- Total lines: 189
- Purpose: Support code file in the Summer 2026 REU folder.

Class	Lines	What it represents
`VFNN_2`	8-107	Old neural-network model class retained from the original project code.
`VFNN`	109-174	Old neural-network model class retained from the original project code.
`PLinear`	176-189	Data/model/helper class.

Function	Lines	What it does
`__init__`	9-61	Helper function.
`forward`	63-107	Helper function.
`__init__`	110-148	Helper function.
`forward`	150-174	Helper function.
`__init__`	184-186	Helper function.
`forward`	188-189	Helper function.

Summer 2026 REU/oracle upper bound/code/rolling_horizon_gurobi_dispatch.py

- Total lines: 506
- Purpose: Support code file in the Summer 2026 REU folder.

Function	Lines	What it does
`load_data`	44-49	Reads data/configuration from CSV/YAML/text files.
`cove_value`	52-63	Computes revenue, COVE, cost, or percentage improvement.
`continuous_baseload`	66-87	Reads data/configuration from CSV/YAML/text files.
`fixed_costs`	90-98	Computes revenue, COVE, cost, or percentage improvement.
`solve_window`	101-199	Builds/solves the optimization problem for one planning window.

`check_constraints`	202-229	Checks feasibility: SoC bounds, grid cap, wind balance, and recursion.
`write_progress`	232-240	Helper function.
`run_rolling`	243-346	Helper function.
`add_compatibility_columns`	349-368	Helper function.
`summarize`	371-419	Computes result rows: revenue, COVE, final SoC, and QA fields.
`main`	422-501	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/oracle upper bound/code/storage.py

- Total lines: 160
- Purpose: Support code file in the Summer 2026 REU folder.

Class	Lines	What it represents
`Storage`	4-24	Storage configuration and derived storage properties.
`BatteryLI`	26-43	Data/model/helper class.
`CAES`	45-56	Data/model/helper class.
`Hydro`	58-69	Data/model/helper class.
`BatteryLA`	71-88	Data/model/helper class.
`BatteryVRF`	90-107	Data/model/helper class.
`Zinc`	109-120	Data/model/helper class.
`Hydrogen`	122-133	Data/model/helper class.
`Gravitational`	135-146	Data/model/helper class.
`Thermal`	148-159	Data/model/helper class.

Function	Lines	What it does
`get_ratings`	5-6	Helper function.
`get_durations`	8-9	Helper function.
`get_rte`	11-12	Helper function.
`get_capex`	14-15	Helper function.
`get_opex`	17-18	Helper function.
`get_rating_index`	20-21	Helper function.
`get_duration_index`	23-24	Helper function.
`__init__`	27-43	Helper function.
`__init__`	46-56	Helper function.
`__init__`	59-69	Helper function.
`__init__`	72-88	Helper function.
`__init__`	91-107	Helper function.
`__init__`	110-120	Helper function.
`__init__`	123-133	Helper function.
`__init__`	136-146	Helper function.
`__init__`	149-159	Helper function.

Summer 2026 REU/oracle upper bound/code/util.py

- Total lines: 368
- Purpose: Support code file in the Summer 2026 REU folder.

Function	Lines	What it does
`get_storage_object`	19-21	Helper function.

`get_rte`	23-25	Helper function.
`get_storage_specs`	27-32	Helper function.
`cove`	37-46	Computes revenue, COVE, cost, or percentage improvement.
`revenue`	48-54	Computes revenue, COVE, cost, or percentage improvement.
`value_factor`	56-59	Helper function.
`batchwise_revenue`	61-63	Computes revenue, COVE, cost, or percentage improvement.
`batchwise_value_factor`	65-69	Helper function.
`batchwise_cove`	71-86	Computes revenue, COVE, cost, or percentage improvement.
`normalize_price`	89-99	Helper function.
`load_config`	101-104	Reads data/configuration from CSV/YAML/text files.
`save_config`	106-108	Helper function.
`load_model`	110-116	Reads data/configuration from CSV/YAML/text files.
`load_model_with_loads`	118-129	Reads data/configuration from CSV/YAML/text files.
`load_dataset_no_split`	131-148	Reads data/configuration from CSV/YAML/text files.
`load_dataset_split_as_tensors`	150-192	Reads data/configuration from CSV/YAML/text files.
`load_dataset_no_split_with_loads`	194-212	Reads data/configuration from CSV/YAML/text files.
`load_dataset`	214-272	Reads data/configuration from CSV/YAML/text files.
`load_dataset_with_loads`	274-328	Reads data/configuration from CSV/YAML/text files.
`load_experiment`	331-341	Reads data/configuration from CSV/YAML/text files.
`plot_losses`	343-360	Creates or formats figures.
`format_num`	362-368	Helper function.

Summer 2026 REU/rolling horizon/EXPERIMENT_KNOBS.py

- Total lines: 52
- Purpose: Stores Step 2 deterministic rolling-horizon knobs.
- Important: Most important knobs: HORIZONS, STORAGE_DURATION_H, EXECUTION_STEP_HOURS, REPLANNING_INTERVAL_HOURS, DIRECT_RESERVE_MW.

Summer 2026 REU/rolling horizon/RUN_2_ROLLING_HORIZON.py

- Total lines: 300
- Purpose: Front-door runner for Step 2 deterministic causal ridge + Gurobi dispatch.
- Important: Builds the command for forecast_backtest_rolling_horizons.py.
- Important: Prints wind-only first and 100 MW benchmark underneath.
- Important: Saves summary CSVs and figures.

Function	Lines	What it does
`add_optional`	44-46	Helper function.
`command`	49-95	Builds the terminal command that calls the deeper runner.

`load_rows`	98-100	Reads data/configuration from CSV/YAML/text files.
`copy_outputs`	103-108	Helper function.
`cove_gain_vs_wind`	111-112	Computes revenue, COVE, cost, or percentage improvement.
`cove_gain_vs_100mw`	115-117	Computes revenue, COVE, cost, or percentage improvement.
`raw_revenue_gain_vs_wind`	120-122	Computes revenue, COVE, cost, or percentage improvement.
`raw_revenue_gain_vs_100mw`	125-127	Computes revenue, COVE, cost, or percentage improvement.
`draw_figures`	130-204	Creates or formats figures.
`main`	207-295	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/rolling horizon/code/compare_rolling_horizons.py

- Total lines: 319
- Purpose: Support code file in the Summer 2026 REU folder.

Function	Lines	What it does
`max_constraint_violation`	28-34	Helper function.
`load_summary`	37-53	Computes result rows: revenue, COVE, final SoC, and QA fields.
`style_axis`	56-60	Creates or formats figures.
`save_performance_figures`	63-146	Creates or formats figures.
`load_example_week`	149-165	Reads data/configuration from CSV/YAML/text files.
`save_example_week_figures`	168-218	Creates or formats figures.
`main`	221-314	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/rolling horizon/code/dataset.py

- Total lines: 44
- Purpose: Support code file in the Summer 2026 REU folder.

Class	Lines	What it represents
`VF2Dataset`	4-25	Old neural-network model class retained from the original project code.
`VFDataset`	27-44	Old neural-network model class retained from the original project code.

Function	Lines	What it does
`__init__`	5-9	Helper function.
`__len__`	12-20	Helper function.
`__getitem__`	23-25	Helper function.
`__init__`	28-31	Helper function.
`__len__`	34-39	Helper function.
`__getitem__`	42-44	Helper function.

Summer 2026 REU/rolling horizon/code/forecast_backtest_rolling_horizons.py

- Total lines: 1315
- Purpose: Main deterministic forecast-driven rolling-horizon engine.
- Important: Fits direct ridge forecast models.
- Important: Builds forecast matrices.
- Important: Runs Gurobi horizon by horizon.
- Important: Executes the planned dispatch against realized wind and price.
- Important: Computes wind-only and 100 MW comparison metrics.

Class	Lines	What it represents
`DirectForecastModel`	56-66	A fitted forecast model used to predict future wind/price/power values.

Function	Lines	What it does
`predict`	63-66	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`calendar_features`	69-83	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`origin_features`	86-98	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`single_origin_features`	101-114	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`fit_direct_models`	117-168	Helper function.
`make_forecast_matrix`	171-196	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`make_known_future_matrix`	199-210	Helper function.
`forecast_metrics`	213-251	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`execute_plan_against_actual`	254-315	Converts the planned action into realized operation using actual wind and current SoC.
`check_realized_constraints`	318-383	Checks feasibility: SoC bounds, grid cap, wind balance, and recursion.
`year_end_soc_targets`	386-400	Helper function.
`annual_soc_qa`	403-423	Checks feasibility: SoC bounds, grid cap, wind balance, and recursion.
`wind_only_delivery`	426-428	Helper function.
`constant_output_100mw_delivery`	431-476	Helper function.
`cost_over_revenue`	479-482	Computes revenue, COVE, cost, or percentage improvement.
`apply_direct_reserve`	485-509	Helper function.
`run_horizon`	512-746	Helper function.
`style_axis`	749-753	Creates or formats figures.
`save_figures`	756-930	Creates or formats figures.
`main`	933-1310	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/rolling horizon/code/model.py

- Total lines: 190
- Purpose: Support code file in the Summer 2026 REU folder.

Class	Lines	What it represents
`VFNN_2`	8-107	Old neural-network model class retained

		from the original project code.
`VFNN`	109-174	Old neural-network model class retained from the original project code.
`PLinear`	176-189	Data/model/helper class.

Function	Lines	What it does
`__init__`	9-61	Helper function.
`forward`	63-107	Helper function.
`__init__`	110-148	Helper function.
`forward`	150-174	Helper function.
`__init__`	184-186	Helper function.
`forward`	188-189	Helper function.

Summer 2026 REU/rolling horizon/code/nora_parameters_and_constraints.py

- Total lines: 100
- Purpose: Small reference file for Nora/Chris storage parameters and constraints.
- Important: Use this when explaining the 100 MW / 10 h CAES setup.

Class	Lines	What it represents
`StorageCase`	15-33	Storage configuration and derived storage properties.

Function	Lines	What it does
`energy_capacity_mwh`	24-25	Helper function.
`min_soc_mwh`	28-29	Helper function.
`mid_soc_mwh`	32-33	Helper function.
`print_summary`	69-95	Computes result rows: revenue, COVE, final SoC, and QA fields.

Summer 2026 REU/rolling horizon/code/rolling_horizon_gurobi_dispatch.py

- Total lines: 506
- Purpose: Older direct rolling-horizon Gurobi script used for full-dataset/oracle style studies.
- Important: Contains explicit Gurobi variables and constraints.
- Important: Useful if Chris asks where the MILP is written.

Function	Lines	What it does
`load_data`	44-49	Reads data/configuration from CSV/YAML/text files.
`cove_value`	52-63	Computes revenue, COVE, cost, or percentage improvement.
`continuous_baseload`	66-87	Reads data/configuration from CSV/YAML/text files.
`fixed_costs`	90-98	Computes revenue, COVE, cost, or percentage improvement.
`solve_window`	101-199	Builds/solves the optimization problem for one planning window.
`check_constraints`	202-229	Checks feasibility: SoC bounds, grid cap, wind balance, and recursion.
`write_progress`	232-240	Helper function.
`run_rolling`	243-346	Helper function.
`add_compatibility_columns`	349-368	Helper function.

`summarize`	371-419	Computes result rows: revenue, COVE, final SoC, and QA fields.
`main`	422-501	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/rolling horizon/code/run_nora_matching_forecast_horizons.py

- Total lines: 773
- Purpose: Earlier Nora-matching forecast horizon runner.
- Important: Kept as supporting code and historical comparison.
- Important: Not the cleanest current front door, but useful for understanding the older storage-baseload comparison.

Class	Lines	What it represents
`DirectForecastModel`	50-60	A fitted forecast model used to predict future wind/price/power values.

Function	Lines	What it does
`predict`	57-60	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`calendar_features`	63-77	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`origin_features`	80-92	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`single_origin_features`	95-107	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`fit_direct_models`	110-148	Helper function.
`make_generation_forecasts`	151-168	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`make_weekly_price_forecasts`	171-177	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`forecast_metrics`	180-207	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`solve_window_nora`	210-257	Builds/solves the optimization problem for one planning window.
`execute_frozen_day`	260-311	Converts the planned action into realized operation using actual wind and current SoC.
`run_forecast_horizon`	314-390	Builds forecast inputs, fits a forecast model, or creates predicted future values.
`revenue`	393-394	Computes revenue, COVE, cost, or percentage improvement.
`annualized_dispatch_cost`	397-400	Computes revenue, COVE, cost, or percentage improvement.
`continuous_baseload`	403-420	Reads data/configuration from CSV/YAML/text files.
`check_realized_constraints`	423-442	Checks feasibility: SoC bounds, grid cap, wind balance, and recursion.
`summarize`	445-481	Computes result rows: revenue, COVE, final SoC, and QA fields.
`validate_nora_week`	484-499	Checks feasibility: SoC bounds, grid cap, wind balance, and recursion.
`make_figures`	502-644	Creates or formats figures.
`main`	647-768	Script entry point: reads knobs/arguments, runs the experiment, writes outputs.

Summer 2026 REU/rolling horizon/code/storage.py

- Total lines: 160
- Purpose: Support code file in the Summer 2026 REU folder.

Class	Lines	What it represents
`Storage`	4-24	Storage configuration and derived storage properties.
`BatteryLI`	26-43	Data/model/helper class.
`CAES`	45-56	Data/model/helper class.
`Hydro`	58-69	Data/model/helper class.
`BatteryLA`	71-88	Data/model/helper class.
`BatteryVRF`	90-107	Data/model/helper class.
`Zinc`	109-120	Data/model/helper class.
`Hydrogen`	122-133	Data/model/helper class.
`Gravitational`	135-146	Data/model/helper class.
`Thermal`	148-159	Data/model/helper class.

Function	Lines	What it does
`get_ratings`	5-6	Helper function.
`get_durations`	8-9	Helper function.
`get_rte`	11-12	Helper function.
`get_capex`	14-15	Helper function.
`get_opex`	17-18	Helper function.
`get_rating_index`	20-21	Helper function.
`get_duration_index`	23-24	Helper function.
`__init__`	27-43	Helper function.
`__init__`	46-56	Helper function.
`__init__`	59-69	Helper function.
`__init__`	72-88	Helper function.
`__init__`	91-107	Helper function.
`__init__`	110-120	Helper function.
`__init__`	123-133	Helper function.
`__init__`	136-146	Helper function.
`__init__`	149-159	Helper function.

Summer 2026 REU/rolling horizon/code/util.py

- Total lines: 368
- Purpose: Support code file in the Summer 2026 REU folder.

Function	Lines	What it does
`get_storage_object`	19-21	Helper function.
`get_rte`	23-25	Helper function.
`get_storage_specs`	27-32	Helper function.
`cove`	37-46	Computes revenue, COVE, cost, or percentage improvement.
`revenue`	48-54	Computes revenue, COVE, cost, or percentage improvement.
`value_factor`	56-59	Helper function.
`batchwise_revenue`	61-63	Computes revenue, COVE, cost, or percentage improvement.
`batchwise_value_factor`	65-69	Helper function.
`batchwise_cove`	71-86	Computes revenue, COVE, cost, or

		percentage improvement.
`normalize_price`	89-99	Helper function.
`load_config`	101-104	Reads data/configuration from CSV/YAML/text files.
`save_config`	106-108	Helper function.
`load_model`	110-116	Reads data/configuration from CSV/YAML/text files.
`load_model_with_loads`	118-129	Reads data/configuration from CSV/YAML/text files.
`load_dataset_no_split`	131-148	Reads data/configuration from CSV/YAML/text files.
`load_dataset_split_as_tensors`	150-192	Reads data/configuration from CSV/YAML/text files.
`load_dataset_no_split_with_loads`	194-212	Reads data/configuration from CSV/YAML/text files.
`load_dataset`	214-272	Reads data/configuration from CSV/YAML/text files.
`load_dataset_with_loads`	274-328	Reads data/configuration from CSV/YAML/text files.
`load_experiment`	331-341	Reads data/configuration from CSV/YAML/text files.
`plot_losses`	343-360	Creates or formats figures.
`format_num`	362-368	Helper function.

The Most Important Equations and Where They Live

Equation / rule	File and lines	Meaning
Wind-only delivery $\text{min}(\text{max}(\text{wind}, 0), \text{grid_cap})$	`rolling horizon/code/forecast_backtest_rolling_hori zons.py:426-428`	No storage; deliver actual wind up to 249 MW.
100 MW benchmark if wind ≥ 100	`100 MW baseload/code/canonical_benchmark_oracle _runner.py:182-214`	Deliver 100 MW, charge excess wind, curtail leftovers.
100 MW benchmark if wind < 100	`100 MW baseload/code/canonical_benchmark_oracle _runner.py:200-210`	Deliver wind, discharge storage toward 100 MW, record shortfall if needed.
SoC update $\text{soc} = \text{soc_start} + \text{charge} - \text{discharge} / \text{rte}$	`100 MW baseload/code/canonical_benchmark_oracle _runner.py:211`	Charging raises storage; discharging lowers stored energy with RTE loss.
Gurobi wind-only charging constraint	`rolling horizon/code/rolling_horizon_gurobi_dispatc h.py:101-199` and `forecast_backtest_rolling_horizons.py:512- 746`	Storage can charge only from wind, not the grid.
Scenario first-hour non-anticipativity	`different scenarios/code/run_uncertainty_aware_disp atch.py:204-210`	All future scenarios must share the first charge/discharge/direct decision.
COVE as cost/revenue	`rolling horizon/code/forecast_backtest_rolling_hori zons.py:479-482`	Lower COVE is better; if storage cost rises faster than revenue, COVE can worsen.

Deep Dive: Step 2 Rolling-Horizon Gurobi

Step 2 is the deterministic dispatch experiment. It uses one forecast path from causal ridge regression, gives it to Gurobi, executes the first 24 hours, carries SoC forward, and repeats.

1. `RUN_2_ROLLING_HORIZON.py:49-95` builds the command. This is where the knobs become command-line flags such as `--storage-duration-h 10.0`, `--execution-step-hours 24`, and `--horizons 24 48 72 168`.
2. `forecast_backtest_rolling_horizons.py:933-1310` parses those flags. The `argparse` parser turns `--grid-cap-mw` into `args.grid_cap_mw`; Python changes the dash name into an underscore attribute.
3. `fit_direct_models` at lines 117-168 trains the ridge forecast models using old data before 2014.
4. `make_forecast_matrix` at lines 171-196 creates the future wind/price matrix for each rolling origin.
5. `run_horizon` at lines 512-746 loops through every planning origin, solves Gurobi, executes the first block, and appends hourly rows.
6. `execute_plan_against_actual` at lines 254-315 is where planned dispatch becomes actual feasible dispatch with actual wind. This is where planned direct wind, actual wind, grid capacity, charge, discharge, and SoC interact.
7. `check_realized_constraints` at lines 318-383 confirms zero or near-zero violations.
8. The summary fields are assembled in `run_horizon` around lines 640-735: revenue, wind-only revenue, 100 MW revenue, COVE, gains, final SoC, and constraint checks.

The reason Step 2 can lose to wind-only on COVE is not that Gurobi failed. Wind-only has no storage cost. The storage case must earn enough extra revenue to overcome both storage capital cost and 55% CAES efficiency loss. Against the 100 MW storage benchmark, Step 2 wins because both cases include storage and Gurobi times energy better.

Deep Dive: Step 3 Scenario Dispatch

Step 3 tests whether one forecast is too fragile. Instead of giving Gurobi one possible future, the code creates several possible futures from forecast residuals. Gurobi chooses a first-hour action that performs well across those futures.

1. `RUN_3_SCENARIO_COMPARISON.py:128-165` builds the scenario command from knobs.
2. `run_uncertainty_aware_dispatch.py:78-99` builds the center forecast from causal ridge wind and daily-lag price.
3. `residual_quantiles` at lines 111-135 measures historical forecast errors and turns them into scenario offsets.
4. `scenario_matrices` at lines 138-153 creates the scenario wind and price matrices.
5. `solve_scenario_window` at lines 156-223 is the Gurobi multi-scenario MILP.
6. Lines 204-210 enforce first-hour non-anticipativity: every scenario must use the same first-hour charge, discharge, direct-wind, and mode decision.
7. `execute_first_hour_storage_action` at lines 226-260 applies that action to actual realized wind for one hour.
8. `run_scenario_controller` at lines 394-455 repeats this across the full backtest period.

Three scenarios currently wins on revenue because it adds enough uncertainty coverage without becoming too conservative. Ten scenarios performs worse because too many extreme futures can make the controller cautious and less profitable.

Deep Dive: Step 4 Oracle

Oracle means perfect future information. It gives Gurobi the actual future wind and actual future price. This is not realistic, but it is useful because it shows the upper bound.

Step 4 now prints two oracle cases:

- Daily-replan oracle: `--execution-step-hours 24 --replanning-interval-hours 24`. This matches the daily ladder style.
- Hourly-replan oracle ceiling: `--execution-step-hours 1 --replanning-interval-hours 1`. This is the more aggressive perfect-information ceiling.

The split is implemented in `RUN_4_ORACLE_UPPER_BOUND.py`:284-338. The function `print_oracle_block` at lines 217-264 prints wind-only first and the 100 MW benchmark underneath.

How to Answer Chris If He Asks “Where Is X?”

Chris asks	Open this file	Point to these lines
Where do I change storage duration?	<code>`rolling horizon/EXPERIMENT_KNOBS.py`</code> or <code>`oracle upper bound/EXPERIMENT_KNOBS.py`</code>	storage block near lines 25-40
Where is the 100 MW benchmark rule?	<code>`100 MW baseload/code/canonical_benchmark_oracle_runner.py`</code>	<code>`run_constant_output_baseload`</code> , lines 182-241
Where is wind-only implemented?	<code>`rolling horizon/code/forecast_backtest_rolling_horizons.py`</code>	<code>`wind_only_delivery`</code> , lines 426-428
Where does Gurobi optimize?	<code>`rolling horizon/code/forecast_backtest_rolling_horizons.py`</code>	<code>`run_horizon`</code> , lines 512-746
Where are constraints checked?	<code>`rolling horizon/code/forecast_backtest_rolling_horizons.py`</code>	<code>`check_realized_constraints`</code> , lines 318-383
Where are scenarios made?	<code>`different scenarios/code/run_uncertainty_aware_dispatch.py`</code>	<code>`residual_quantiles`</code> and <code>`scenario_matrices`</code> , lines 111-153
Where is first-hour scenario non-anticipativity?	<code>`different scenarios/code/run_uncertainty_aware_dispatch.py`</code>	lines 204-210
Where does Step 4 split daily and hourly oracle?	<code>`oracle upper bound/RUN_4_ORACLE_UPPER_BOUND.py`</code>	<code>`main`</code> , lines 284-338
Where does COVE get calculated?	<code>`rolling horizon/code/forecast_backtest_rolling_horizons.py`</code>	<code>`cost_over_revenue`</code> , lines 479-482 and summary assembly in <code>`run_horizon`</code>

What to Run in Each Folder

```
cd "/Users/davidvalenta/deep-wind-model-dispatch/Summer 2026 REU/100 MW baseload"
../../venv/bin/python RUN_0_100MW_BASELOAD.py
```

```
cd "/Users/davidvalenta/deep-wind-model-dispatch/Summer 2026 REU/causal ridge regression"
../../venv/bin/python RUN_1_FORECAST_RMSE.py
```

```
cd "/Users/davidvalenta/deep-wind-model-dispatch/Summer 2026 REU/rolling horizon"
../../venv/bin/python RUN_2_ROLLING_HORIZON.py
```

```
cd "/Users/davidvalenta/deep-wind-model-dispatch/Summer 2026 REU/different scenarios"
../../venv/bin/python RUN_3_SCENARIO_COMPARISON.py
```

```
cd "/Users/davidvalenta/deep-wind-model-dispatch/Summer 2026 REU/oracle upper bound"
```

```
../../venv/bin/python RUN_4_ORACLE_UPPER_BOUND.py
```

Final Mental Model

The code is not one single giant program. It is five small front doors that call deeper scripts. The front doors are the files you run. The knobs files are where you change settings. The code folders contain the math. The results folders contain the proof. If you get lost, start from the runner for that step and follow the SOURCE_RUNNER or RUNNER variable into the code/ folder.